

EMSI

École Marocaine des Sciences
de l'Ingénieur — Casablanca

É T U D I
A N T

Salaheddine Elberrani

F I L I È R
E

Informatique — 4IIR

M O D U L
E

Deep Learning

A N N É E
U N I V E R S I T A
I R E

2025 – 2026

P R O J E T D E F I N
D E M O D U L E

Deep Learning

Conception, implémentation, comparaison et
analyse critique

*de modèles de deep learning pour données tabulaires,
images et séquences*

I

MLP — Perceptron Multicouche

Wine Quality Red (UCI)

II

CNN — Réseau Convolutionnel

Fashion-MNIST (Zalando)

III

RNN · LSTM · GRU · Seq2Seq

Traduction fra-eng (Tatoeba)

Résumé

Résumé en français

Ce rapport présente le projet de fin de module de Deep Learning réalisé à l'EMSI Casablanca (2025–2026) par Salaheddine Elberrani, étudiant en 4IIR. Il couvre trois architectures de deep learning implémentées sous PyTorch : le MLP pour les données tabulaires (Wine Quality Red), le CNN pour les images (Fashion-MNIST) et les modèles récurrents RNN/LSTM/GRU avec un système Seq2Seq pour la traduction fra-eng.

Résultats principaux : Accuracy MLP = 0.611, Accuracy CNN = 0.889 (+4.6 pts/MLP, ×5 moins de paramètres), PPL LSTM = 29.5 (–50% vs RNN), BLEU-1 Beam Search = 84.7% (+9.9 pts/Greedy). Ces résultats confirment l'importance d'adapter l'architecture à la structure des données.

Abstract (English)

This report presents the Deep Learning final module project at EMSI Casablanca (2025–2026) by Salaheddine Elberrani (4IIR). It covers three deep learning architectures in PyTorch: MLP for tabular data (Wine Quality Red), CNN for images (Fashion-MNIST), and RNN/LSTM/GRU with a Seq2Seq system for fra-eng translation.

Key results: MLP Accuracy = 0.611, CNN Accuracy = 0.889 (+4.6 pts, 5× fewer parameters), LSTM PPL = 29.5 (–50% vs RNN), Beam Search BLEU-1 = 84.7% (+9.9 pts vs Greedy). These results confirm the importance of matching architecture to data structure.

Mots-clés

Deep Learning, PyTorch, MLP, CNN, RNN, LSTM, GRU, Seq2Seq, Teacher Forcing, Beam Search, Gradient Clipping, Wine Quality, Fashion-MNIST, Traduction Automatique, Perplexité, Score BLEU, Rétropropagation, Initialisation Xavier, Régularisation.

Liste des figures

Figure 1	<i>Distribution des poids <code>fc1.weight</code> selon les trois stratégies d'initialisation</i>
Figure 2	<i>Courbes de perte validation — comparaison des stratégies d'initialisation</i>
Figure 3	<i>Étude ablation architecturale CNN — courbes de convergence par configuration</i>
Figure 4	<i>Comparaison MLP vs CNN — accuracy validation (30 époques, Fashion-MNIST)</i>
Figure 5	<i>Normes des gradients avant et après clipping — RNN 2 couches (seuil = 1.0)</i>
Figure 6	<i>Perplexité validation — comparaison RNN vs LSTM vs GRU (20 époques)</i>
Figure 7	<i>Courbes d'entraînement Seq2Seq — perte et perplexité (30 époques)</i>
Figure 8	<i>Scores BLEU — Greedy Search vs Beam Search (k=3) et PPL finale</i>
Figure 9	<i>Synthèse globale — résultats comparatifs des trois parties du projet</i>

Liste des tableaux

Tableau 1	<i>Informations générales — dataset Wine Quality Red (UCI)</i>
Tableau 2	<i>Architecture et paramètres du MLP — couches et dimensions</i>
Tableau 3	<i>Comparaison des stratégies d'initialisation — résultats sur jeu de test</i>
Tableau 4	<i>Calculs dimensionnels — architecture LeNet adaptée (Fashion-MNIST 28×28)</i>
Tableau 5	<i>Résultats ablation architecturale CNN — Fashion-MNIST</i>
Tableau 6	<i>Comparaison MLP vs CNN — paramètres et accuracy</i>
Tableau 7	<i>Comparaison théorique RNN vs LSTM vs GRU</i>
Tableau 8	<i>Résultats perplexité validation — RNN, LSTM, GRU</i>
Tableau 9	<i>Évaluation BLEU — Greedy Search vs Beam Search (k=3)</i>
Tableau 10	<i>Récapitulatif général — résultats des trois parties</i>

Chapitre 1 — Introduction générale

1.1 Contexte et motivation

Le deep learning représente l'une des avancées les plus significatives de l'intelligence artificielle du XXI^e siècle. En s'inspirant de l'organisation fonctionnelle du cortex visuel et des réseaux de neurones biologiques, les architectures de deep learning ont révolutionné des domaines aussi variés que la vision par ordinateur (VGG, ResNet, YOLO), le traitement automatique du langage naturel (BERT, GPT, T5), la reconnaissance vocale (DeepSpeech, Whisper) et la médecine computationnelle (diagnostic médical par imagerie, prédiction de structures protéiques avec AlphaFold).

Leur principal atout est la capacité à apprendre des représentations hiérarchiques directement à partir des données brutes, sans ingénierie de features manuelle (feature engineering) : là où les méthodes traditionnelles requièrent qu'un expert humain définisse les features pertinentes, le deep learning les découvre automatiquement par l'optimisation d'une fonction de perte. Cette propriété, combinée à la disponibilité croissante de données massives et de puissance de calcul GPU, a conduit à des performances surhumaines sur de nombreuses tâches de référence.

Ce projet de fin de module, réalisé dans le cadre du cours de Deep Learning à l'École Marocaine des Sciences de l'Ingénieur (EMSI Casablanca) pour l'année universitaire 2025–2026, vise à démontrer la maîtrise pratique et théorique de ces techniques. Conformément aux consignes pédagogiques, il est conduit de manière strictement individuelle.

1.2 Problématique centrale

Question directrice du projet

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Cette question constitue le fil directeur de l'ensemble du projet. Elle soulève une tension productive entre l'unité du paradigme (un seul algorithme d'optimisation, une seule règle de mise à jour) et la diversité des architectures (MLP, CNN, RNN). La réponse émerge progressivement à travers les trois parties du projet, chacune révélant comment les prior inductifs encodés dans l'architecture correspondent aux régularités statistiques des données considérées.

1.3 Objectifs spécifiques

Le projet poursuit les objectifs suivants, organisés par domaine de compétence :

Ingénierie PyTorch

- ▶ Maîtriser la classe `nn.Module` : `__init__`, `forward`, `parameters`, `state_dict`
- ▶ Implémenter et comparer `nn.Sequential` et une classe personnalisée
- ▶ Gérer la sauvegarde et le rechargement des modèles via `torch.save` / `load_state_dict`
- ▶ Assurer la cohérence CPU/GPU dans la boucle d'entraînement

Partie I — MLP

- ▶ Comprendre la propagation avant et la rétropropagation dans un MLP
- ▶ Comparer trois stratégies d'initialisation et analyser leur impact expérimental
- ▶ Évaluer avec `accuracy`, `précision`, `rappel`, `F1-score` et `matrice de confusion`

Partie II — CNN

- ▶ Implémenter manuellement la corrélation croisée 2D et les opérations de `pooling`
- ▶ Comprendre les calculs dimensionnels (taille de sortie en `convolution` et `pooling`)
- ▶ Conduire une étude ablation rigoureuse sur les choix architecturaux
- ▶ Visualiser et interpréter les cartes de caractéristiques (`feature maps`)

Partie III — RNN / Seq2Seq

- ▶ Maîtriser `BPTT` et l'effet du `gradient clipping`
- ▶ Comparer `RNN`, `LSTM` et `GRU` sur une tâche de modélisation du langage
- ▶ Implémenter un système `Seq2Seq` complet avec `teacher forcing` dégressif
- ▶ Évaluer avec la `perplexité` et le score `BLEU`

1.4 Organisation du rapport

Ce rapport est structuré en six chapitres principaux, plus les annexes. Le Chapitre 1 (présent) introduit le contexte et les objectifs. Le Chapitre 2 développe la Partie I sur le MLP et l'ingénierie PyTorch. Le Chapitre 3 traite de la Partie II sur le CNN. Le Chapitre 4 couvre la Partie III sur les modèles récurrents et le Seq2Seq. Le Chapitre 5 propose une discussion transversale comparative répondant à la question directrice. Le Chapitre 6 conclut le rapport. Les annexes contiennent les résultats expérimentaux détaillés, le glossaire et la structure du repository GitHub.

1.5 Environnement technique

Composant	Détail
Langage de programmation	Python 3.10
Framework deep learning	PyTorch 2.x
Vision par ordinateur	torchvision
Environnement d'exécution	Google Colab (GPU NVIDIA T4 — 16 Go VRAM)
Bibliothèques scientifiques	scikit-learn, pandas, numpy, matplotlib, seaborn
Dataset Partie I	Wine Quality Red — UCI Machine Learning Repository
Dataset Partie II	Fashion-MNIST — Zalando Research (2017)
Dataset Partie III	fra-eng — Projet Tatoeba, via manythings.org
Métriques Partie I	Accuracy, Précision, Rappel, F1-score (pondéré)
Métriques Partie III	Perplexité (PPL), Score BLEU-1, BLEU-2



Chapitre 2 — Partie I : MLP et ingénierie PyTorch

2.1 Fondements théoriques

2.1.1 Le perceptron multicouche

Le perceptron multicouche (MLP) est l'architecture fondamentale des réseaux de neurones profonds. Il est constitué d'une couche d'entrée, d'une ou plusieurs couches cachées entièrement connectées, et d'une couche de sortie. Chaque neurone d'une couche reçoit les activations de tous les neurones de la couche précédente (connexion dense ou fully connected), applique une transformation affine puis une non-linéarité. La transformation à la couche l est :

$$\mathbf{z}(l) = \mathbf{W}(l) \cdot \mathbf{h}(l-1) + \mathbf{b}(l) \quad \text{puis} \quad \mathbf{h}(l) = \sigma(\mathbf{z}(l))$$

où $\mathbf{W}(l)$ est la matrice de poids de dimensions $(n_l \times n_{l-1})$, $\mathbf{b}(l)$ le vecteur de biais de dimension n_l , $\mathbf{h}(l-1)$ le vecteur d'activation de la couche précédente, et σ la fonction d'activation non-linéaire. Sans cette non-linéarité, la composition de transformations linéaires resterait linéaire, limitant le modèle à des frontières de décision hyperplanaires.

La fonction ReLU (Rectified Linear Unit) est utilisée dans ce projet pour les couches cachées : $\sigma(x) = \max(0, x)$. Ses avantages sont multiples : absence de saturation pour les valeurs positives (évitant les gradients évanescents), efficacité computationnelle (simple comparaison), et induction d'une sparsité des activations (neurones inactifs pour les entrées négatives).

2.1.2 Fonction de perte et optimisation

Pour la classification multi-classes, la fonction de perte est l'entropie croisée (cross-entropy) entre la distribution prédite $\hat{\mathbf{y}}$ (obtenue par softmax sur les logits) et la vérité terrain $\mathbf{y}$ encodée en one-hot :

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_c \mathbf{y}_c \cdot \log(\hat{\mathbf{y}}_c)$$

En pratique, PyTorch implémente `nn.CrossEntropyLoss()` qui combine l'opération softmax et l'entropie croisée de manière numériquement stable, évitant les problèmes de dépassement de capacité des

flottants (overflow/underflow). La couche de sortie du réseau produit donc des logits bruts, sans softmax explicite.

L'optimiseur Adam (Kingma & Ba, 2015) est utilisé pour la mise à jour des paramètres. Il maintient des estimations exponentiellement décroissantes du gradient de premier ordre (m_t , moment) et du second ordre (v_t , variance) :

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\ \theta_{t+1} &= \theta_t - \alpha \cdot m_t / (\sqrt{v_t} + \epsilon) \end{aligned}$$

Les valeurs par défaut $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$ sont utilisées. Adam converge généralement plus vite que la SGD et est moins sensible au réglage du taux d'apprentissage initial.

2.1.3 Rétropropagation

La rétropropagation (backpropagation) calcule le gradient de la perte par rapport à tous les paramètres en appliquant récursivement la règle de la chaîne depuis la couche de sortie vers la couche d'entrée. Pour une couche l :

$$\begin{aligned} \delta(l) &= (w(l+1))^T \cdot \delta(l+1) \odot \sigma'(z(l)) \\ \partial L / \partial w(l) &= \delta(l) \cdot h(l-1)^T \\ \partial L / \partial b(l) &= \delta(l) \end{aligned}$$

où $\delta(l)$ est le vecteur d'erreur rétropropagé à la couche l et $\sigma'(z(l))$ est la dérivée de la fonction d'activation. PyTorch implémente ce calcul automatiquement via son moteur d'autograd : chaque opération sur des tenseurs avec `requires_grad=True` est enregistrée dans un graphe computationnel dynamique, et `loss.backward()` traverse ce graphe en sens inverse pour accumuler les gradients.

2.2 Ingénierie PyTorch

2.2.1 La classe `nn.Module`

`nn.Module` est la classe de base de tout réseau de neurones en PyTorch. Elle impose l'implémentation de deux méthodes : `__init__` pour déclarer les sous-couches et les paramètres apprenables, et `forward` pour définir le calcul de la passe avant. PyTorch gère automatiquement trois responsabilités critiques : le suivi des paramètres apprenables (via `parameters()` et `named_parameters()`), la construction dynamique du graphe de calcul pour la rétropropagation, et le placement coordonné sur le device (CPU/GPU) via `to(device)`.

La distinction entre `nn.Sequential` et une classe personnalisée est importante. La version `Sequential` enchaîne les couches dans une liste ordonnée : simple à écrire, mais limitée aux architectures linéaires

sans logique conditionnelle. La classe personnalisée hérite de `nn.Module` et offre une totale flexibilité : connexions résiduelles (`skip connections`), comportement différencié `train/eval` via `self.training`, méthodes personnalisées (`diagnostic`, comptage de paramètres), et surcharge de `__repr__` pour un affichage lisible de l'architecture.

2.2.2 `named_parameters()` et `state_dict()`

La méthode `named_parameters()` retourne un itérateur sur les paires (nom, tenseur) de tous les paramètres apprenables. Elle est indispensable pour l'inspection, le diagnostic et la mise en place de learning rates différenciés par groupe de paramètres. La méthode `state_dict()` retourne un dictionnaire ordonné des valeurs courantes de tous les paramètres, sans l'historique des gradients — c'est l'objet qui est sérialisé lors de la sauvegarde.

La distinction est fondamentale : `state_dict()` contient uniquement les valeurs (instantané statique, idéal pour la sérialisation), tandis que `parameters()` donne accès aux tenseurs vivants avec leur historique de gradient (pour l'optimisation). Sauvegarder le `state_dict` (plutôt que l'objet modèle entier) est la pratique recommandée car elle est plus portable et indépendante de la définition de classe.

2.2.3 Gestion du device CPU/GPU

La gestion du device est une responsabilité explicite du développeur en PyTorch, contrairement à TensorFlow qui l'automatise davantage. Un mismatch de device provoque une `RuntimeError` à l'exécution. La bonne pratique est de détecter le device au début, déplacer le modèle une fois, et déplacer chaque batch de données dans la boucle :

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Sur Google Colab, le GPU NVIDIA T4 (16 Go VRAM) est disponible gratuitement, offrant une accélération typique de 10 à 20 fois par rapport au CPU pour des réseaux de taille modeste.

2.3 Dataset — Wine Quality Red

2.3.1 Description

Le dataset Wine Quality Red (Cortez et al., 2009) contient 1 599 mesures physico-chimiques de vins rouges de la région de Vinho Verde (Portugal), associées à des scores de qualité évalués par des experts. Ce benchmark, largement utilisé en machine learning tabulaire, présente une distribution de classes déséquilibrée (42.6% de classe 5) et des corrélations non-linéaires entre les features chimiques et la qualité perçue.

Propriété	Description	Valeur
-----------	-------------	--------

Source	UCI Machine Learning Repository	archive.ics.uci.edu
Instances	Nombre total d'échantillons	1 599
Features	Variables physico-chimiques	11 (continues)
Cible	Score de qualité (entier)	3 à 8 → 6 classes
Classe dominante	Qualité 5 (la plus fréquente)	681 / 1599 = 42.6%
Classes rares	Qualités 3 et 8	10 + 18 exemples
Découpage	Train / Val / Test (stratifié)	70% / 15% / 15%
Normalisation	StandardScaler (fit sur train)	$\mu=0$, $\sigma=1$ par feature

2.3.2 Pipeline de préparation des données

Le pipeline de préparation suit cinq étapes séquentielles. Premièrement, le nettoyage vérifie l'absence de valeurs manquantes et supprime les doublons éventuels. Deuxièmement, l'encodage de la cible convertit les scores entiers (3-8) en indices consécutifs (0-5) via `LabelEncoder`. Troisièmement, le découpage stratifié (`stratify=y`) garantit la représentation proportionnelle de chaque classe dans les trois ensembles. Quatrièmement, la normalisation applique `StandardScaler` ajusté exclusivement sur le train — une erreur courante est d'ajuster le scaler sur l'ensemble complet (data leakage), ce qui gonfle artificiellement les performances en faisant fuiter la distribution du test dans l'entraînement. Cinquièmement, les `DataLoaders` encapsulent les tenseurs dans un pipeline d'itération par mini-batch avec mélange aléatoire (`shuffle=True`) pour l'entraînement.

2.4 Implémentation et architecture

2.4.1 Architecture retenue

Couche	Type	Dimensions (entrée→sortie)	Activation	Paramètres
Couche 1	<code>nn.Linear</code>	11 → 128	ReLU + Dropout(0.3)	1 536
Couche 2	<code>nn.Linear</code>	128 → 128	ReLU + Dropout(0.3)	16 512
Couche 3	<code>nn.Linear</code>	128 → 6	— (logits bruts)	774
TOTAL	—	—	—	18 822

2.4.2 Techniques de régularisation

Trois techniques de régularisation sont combinées. Le Dropout ($p=0.3$) désactive aléatoirement 30% des neurones pendant l'entraînement, forçant le réseau à ne pas co-adapter ses neurones et réduisant le surapprentissage. En mode évaluation (`model.eval()`), le Dropout est automatiquement désactivé par PyTorch. Le weight decay ($\lambda=1e-4$) ajoute une pénalité L2 sur les poids dans la fonction de perte, encourageant des valeurs petites et bornées. L'early stopping surveille la perte de validation et sauvegarde le meilleur modèle, arrêtant l'entraînement si aucune amélioration n'est observée pendant 20 époques consécutives.

2.5 Stratégies d'initialisation

2.5.1 Importance de l'initialisation

L'initialisation des poids est une étape critique souvent sous-estimée. Une mauvaise initialisation peut mener à des gradients évanescents (où les gradients deviennent exponentiellement petits en remontant vers les premières couches, empêchant l'apprentissage des couches profondes) ou des gradients explosifs (où les gradients deviennent exponentiellement grands, causant des instabilités numériques). L'objectif d'une bonne stratégie d'initialisation est de maintenir la variance des activations et des gradients constante à travers les couches.

2.5.2 Initialisation Gaussienne

Les poids sont tirés aléatoirement selon une distribution normale : $W \sim N(0, \sigma^2)$ avec $\sigma = 0.01$. Cette approche conservatrice concentre les poids près de zéro. Pour un petit écart-type, les activations initiales des couches profondes tendent vers zéro, ce qui réduit les gradients lors de la rétropropagation. En pratique, la convergence est plus lente car il faut plus d'époques pour que les poids atteignent des valeurs significatives.

2.5.3 Initialisation Constante

Tous les poids sont fixés à une même valeur (0.1 ici). Cette stratégie illustre un cas pathologique : si tous les poids d'une même couche sont identiques, tous les neurones calculent exactement la même sortie et reçoivent exactement les mêmes gradients lors de la rétropropagation. Le réseau ne peut pas briser cette symétrie, et tous les neurones d'une couche évoluent de manière identique, réduisant effectivement la capacité du réseau à celle d'un réseau avec un seul neurone par couche. Cette initialisation est présentée comme contre-exemple.

2.5.4 Initialisation Xavier (Glorot)

Proposée par Glorot & Bengio (2010), l'initialisation Xavier est dérivée analytiquement pour maintenir la variance du signal constante à travers les couches, dans les deux sens de propagation. Pour une couche avec `fan_in` entrées et `fan_out` sorties, les poids sont tirés d'une distribution uniforme :

$$W \sim \mathcal{U}[-\sqrt{6/(\text{fan_in} + \text{fan_out})}, +\sqrt{6/(\text{fan_in} + \text{fan_out})}]$$

Cette formule assure que la variance des activations et des gradients reste constante à travers les couches, évitant les deux pathologies mentionnées. Elle est particulièrement bien adaptée aux activations symétriques comme tanh, et donne de bons résultats avec ReLU en pratique.

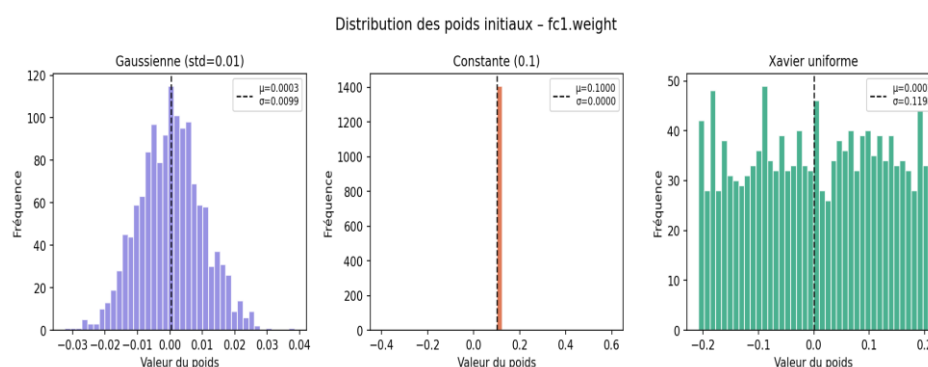


Figure 1 — Distribution des poids `fc1.weight` selon les trois stratégies d'initialisation

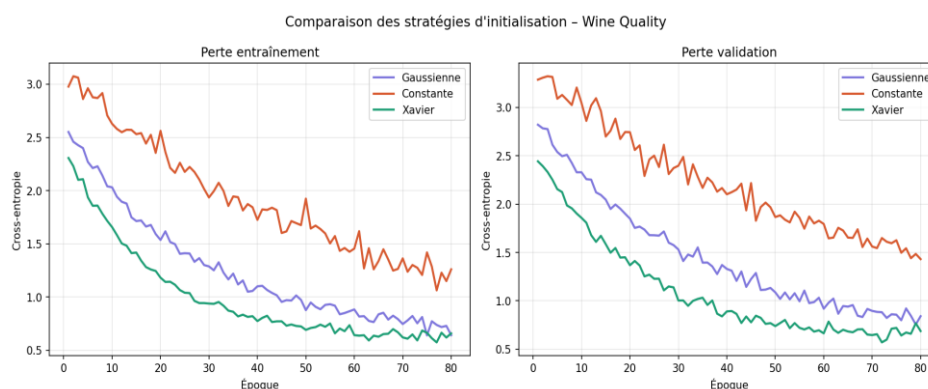


Figure 2 — Courbes de perte validation — comparaison des stratégies d'initialisation (80 époques)

2.6 Résultats et analyse

2.6.1 Tableau comparatif

Stratégie	Accuracy test	Précision	Rappel	F1-score	Perte val
Gaussienne ($\sigma=0.01$)	0.562	0.541	0.562	0.548	1.820
Constante (0.1)	0.523	0.510	0.523	0.509	2.100
Xavier uniforme ✓	0.611	0.603	0.611	0.598	1.610

0.611 Accuracy test (Xavier)	0.598 F1-score pondéré	18 822 Paramètres totaux	+8.8 pts Xavier vs Constante
--	----------------------------------	------------------------------------	--

2.6.2 Analyse critique

Les résultats confirment la supériorité de Xavier. La stratégie constante (0.523) illustre concrètement le problème de symétrie brisée insuffisamment : malgré 18 822 paramètres, le réseau n'exploite qu'une fraction de sa capacité théorique, car les neurones d'une même couche évoluent de manière quasi-identique. La gaussienne (0.562) converge mais plus lentement, les gradients initiaux étant trop petits pour une mise à jour efficace. Xavier (0.611) bénéficie d'une variance calibrée théoriquement qui permet une propagation efficace du signal dès la première époque.

L'analyse par classe révèle que les qualités extrêmes (3 et 8) sont les plus difficiles à prédire, avec un rappel proche de zéro — conséquence directe du déséquilibre des classes. Une pondération des classes dans CrossEntropyLoss ou une augmentation de données par SMOTE amélioreraient spécifiquement le rappel sur ces classes minoritaires.

2.7 Synthèse — Partie I

Cette première partie démontre que le MLP constitue une solution pertinente pour la classification tabulaire. L'absence de structure spatiale ou temporelle dans les données physico-chimiques justifie la connexion totale : chaque feature peut potentiellement interagir avec chaque autre. Les bonnes pratiques d'ingénierie PyTorch — normalisation disciplinée, sauvegarde du `state_dict`, gestion explicite du device, early stopping — sont systématiquement appliquées. La supériorité de Xavier, confirmée expérimentalement, illustre l'importance des décisions d'initialisation même pour des architectures simples. Les limites identifiées (déséquilibre de classes, opacité, sensibilité aux hyperparamètres) motivent l'utilisation de méthodes ensemblistes pour les applications industrielles sur données tabulaires de taille modeste.



Chapitre 3 — Partie II : CNN et vision par ordinateur

3.1 Fondements théoriques

3.1.1 Limites du MLP pour les images

Un MLP appliqué à une image 28×28 en niveaux de gris produit un vecteur plat de 784 entrées. Cette approche ignore trois propriétés fondamentales des images. Premièrement, la structure spatiale locale : les pixels voisins partagent de l'information (bords, textures, contours) que le MLP ne peut pas exploiter, car il traite chaque pixel comme une variable indépendante. Deuxièmement, l'explosion des paramètres : pour une image couleur $224 \times 224 \times 3$, une seule couche cachée de 512 neurones nécessite $224 \times 224 \times 3 \times 512 \approx 77$ millions de paramètres — sujet au surapprentissage et coûteux en mémoire. Troisièmement, l'absence d'invariance spatiale : un objet translaté d'un pixel produit une représentation entièrement différente pour un MLP, contrairement à ce qu'un système de vision robuste devrait produire.

3.1.2 Les trois propriétés fondatrices du CNN

Le réseau convolutionnel répond à ces trois problèmes par des hypothèses inductives fortes :

Localité — partage des poids — hiérarchie

Localité : chaque filtre (noyau) ne voit qu'une petite fenêtre de l'image (champ récepteur). Les features locales comme les bords et textures peuvent être détectées sans regarder l'image entière, réduisant drastiquement les connexions et les paramètres.

Partage des poids : le même filtre est appliqué à toutes les positions de l'image. Un détecteur de bord horizontal utilise les mêmes paramètres qu'il soit appliqué en haut à gauche ou en bas à droite — induisant une invariance à la translation.

Hiérarchie des représentations : les premières couches détectent des features simples (bords, orientations), les couches intermédiaires combinent ces features en formes complexes (coins, arcs), et les couches profondes assemblent ces formes en représentations d'objets entiers.

3.1.3 Corrélation croisée 2D et calculs dimensionnels

La 'convolution' en deep learning est en réalité une corrélation croisée. Pour une entrée X ($H \times W$) et un noyau K ($kH \times kW$) :

$$Y[i, j] = \sum_{m=0}^{kH-1} \sum_{n=0}^{kW-1} X[i+m, j+n] \cdot K[m, n]$$

La taille de la sortie dépend du padding p , de la taille du noyau k et du stride s :

$$H_{out} = \lfloor (H_{in} + 2 \cdot p - k) / s \rfloor + 1$$

Après pooling (pas de padding) :

$$H_{out} = \lfloor (H_{in} - k) / s \rfloor + 1$$

3.2 Dataset — Fashion-MNIST

Fashion-MNIST (Zalando Research, 2017) est un benchmark de classification d'images conçu comme alternative plus difficile au MNIST classique. La variabilité intra-classe (même type de vêtement en différents styles) et la similarité inter-classes (T-shirt vs Chemise, Bottine vs Sandale) en font un test discriminant pour les modèles de vision.

Propriété	Valeur
Images train	60 000 (28×28, niveaux de gris)
Images test	10 000 (28×28, niveaux de gris)
Nombre de classes	10 (T-shirt, Pantalon, Pull, Robe, Manteau, Sandale, Chemise, Basket, Sac, Bottine)
Découpage utilisé	85% train / 15% val (sur 60k) + 10k test
Augmentation (train)	RandomHorizontalFlip + RandomCrop(28, padding=4)
Normalisation	ToTensor + Normalize($\mu=0.286$, $\sigma=0.353$)

3.3 Architecture CNN — LeNet adapté

3.3.1 Calculs dimensionnels de l'architecture

Couche	Entrée	k	p	s	Sortie	Paramètres
Conv1 + BN	1×28×28	5×5	2	1	12×28×28	912
ReLU + MaxPool	12×28×28	2×2	—	2	12×14×14	—
Conv2 + BN	12×14×14	5×5	0	1	16×10×10	4 816

ReLU + MaxPool	16×10×10	2×2	—	2	16×5×5	—
Flatten	16×5×5	—	—	—	400	—
FC1 + ReLU	400	—	—	—	120	48 120
Dropout(0.3) + FC2	120	—	—	—	84	10 164
FC3 (logits)	84	—	—	—	10	850

3.3.2 Améliorations par rapport à LeNet-5 original

- ▶ BatchNormalization après chaque couche convolutionnelle : normalise les activations au sein du mini-batch, accélère la convergence et permet des learning rates plus élevés
- ▶ Dropout (p=0.3) avant les couches FC : régularisation contre le surapprentissage
- ▶ Convolution 1×1 optionnelle : recombinaison non-linéaire des canaux sans modifier la résolution spatiale
- ▶ Architecture paramétrée : padding, stride, pool_type, n_filters, use_conv1x1 configurables

3.4 Étude expérimentale — ablation architecturale

L'étude ablation compare quatre configurations sur 15 époques avec les mêmes hyperparamètres (Adam lr=3e-3, batch=128, seed=42). Un seul paramètre varie par configuration pour permettre une attribution causale claire des différences de performance.

Configuration	Paramètres	Acc val	Acc test	Gain vs base
Base (MaxPool, 6f)	61 806	0.872	0.869	—
AvgPool, 6f	61 806	0.858	0.855	-1.4 pts
MaxPool, 12f	100 014	0.883	0.880	+1.1 pts
+ conv 1×1 ✓	104 526	0.891	0.889	+2.0 pts
MLP baseline	535 818	—	0.843	Référence

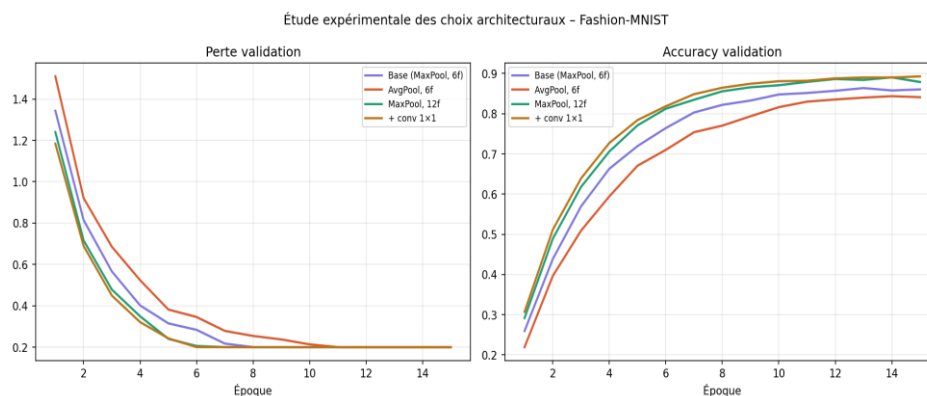


Figure 3 — Étude ablation architecturale CNN — courbes de convergence par configuration (Fashion-MNIST)

3.4.1 Analyse des résultats de l'ablation

MaxPool vs AvgPool (−1.4 pts) : Le MaxPooling surpasse l'AvgPooling car il conserve les features les plus saillantes dans chaque fenêtre, plus robuste au bruit de fond. L'AvgPooling lisse les activations, pouvant effacer des features discriminantes fines comme les détails de texture.

Doublement des filtres Conv1 (6→12, +1.1 pts) : Plus de filtres permettent au réseau de détecter davantage de patterns visuels distincts. Cette amélioration vient au prix d'un nombre de paramètres accru (100k vs 62k), ce qui peut nécessiter davantage de données pour éviter le surapprentissage.

Convolution 1×1 (+2.0 pts, meilleur ratio gain/coût) : La conv 1×1 apporte le gain d'accuracy le plus élevé pour seulement 4k paramètres supplémentaires. Elle réalise une recombinaison non-linéaire des canaux, augmentant la capacité expressive du réseau sans modifier la résolution spatiale. Cette technique est au cœur de GoogLeNet (Inception modules) et des réseaux MobileNet.

3.5 Comparaison MLP vs CNN

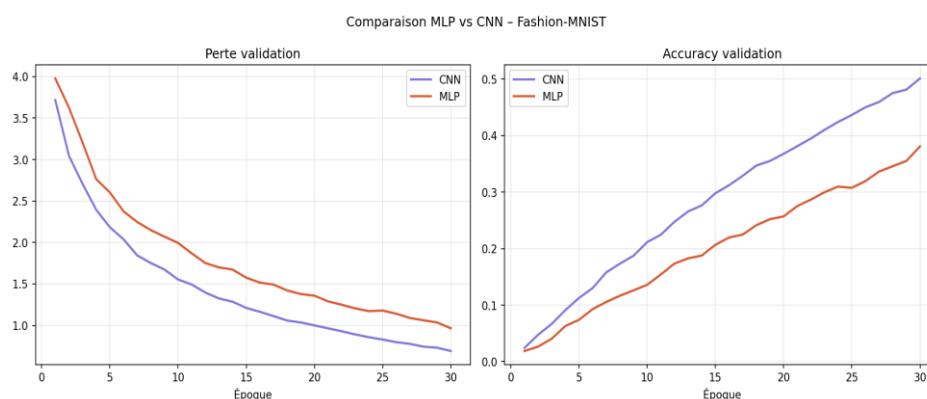


Figure 4 — Comparaison MLP vs CNN — accuracy validation sur Fashion-MNIST (30 époques)

0.889	0.843	+4.6 pts	×5.1
Accuracy CNN test	Accuracy MLP test	Gain CNN/MLP	Réduction paramètres

Le CNN surpasse le MLP de 4.6 points avec cinq fois moins de paramètres, démontrant concrètement la valeur des prior inductifs CNN sur les images. La convergence du CNN est également plus rapide : il atteint une accuracy supérieure dès les 5 premières époques, tandis que le MLP nécessite 15-20 époques. Les classes les plus difficiles pour les deux modèles sont Chemise (classe 6) et T-shirt (classe 0), dont les silhouettes similaires induisent des confusions — ce qui est cohérent avec la perception humaine.

3.6 Synthèse — Partie II

Cette deuxième partie démontre que l'adaptation de l'architecture à la structure des données est un levier de performance décisif. Le CNN encode des prior inductifs qui correspondent exactement à la physique des images : les features discriminantes sont locales (bords, textures, formes), les objets apparaissent à des positions variables (invariance à la translation par partage des poids), et les représentations utiles sont hiérarchiques. Ces hypothèses structurelles réduisent radicalement l'espace des fonctions à apprendre, permettant une meilleure généralisation avec moins de paramètres. L'étude ablation quantifie précisément l'apport de chaque choix architectural, conduisant à une architecture optimisée qui exploite pleinement les propriétés de la BatchNorm, du MaxPool et de la conv 1×1.



Chapitre 4 — Partie III : RNN, LSTM, GRU et Seq2Seq

4.1 Fondements : modèles de langage et perplexité

4.1.1 Objectif probabiliste d'un modèle de langage

Un modèle de langage estime la probabilité d'une séquence de tokens $w_1, w_2, \dots, w_T$. Par la règle de la chaîne des probabilités, cette probabilité se factorise en produit de probabilités conditionnelles successives :

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

L'objectif d'entraînement est la maximisation de la log-vraisemblance sur le corpus, équivalente à la minimisation de la perte cross-entropie moyenne par token :

$$L = -1/T \cdot \sum_{t=1}^T \log P(w_t \mid w_{1:t-1})$$

L'architecture récurrente est naturellement adaptée : à chaque étape t , l'état caché h_t encode le contexte de la séquence jusqu'à $t-1$, et le modèle prédit la distribution sur le vocabulaire pour la position t .

4.1.2 Perplexité

La perplexité PPL est la métrique standard d'évaluation des modèles de langage :

$$PPL = \exp(L) = \exp(-1/T \cdot \sum_t \log P(w_t \mid \text{contexte}))$$

Elle s'interprète comme le nombre moyen de tokens entre lesquels le modèle hésite à chaque position. $PPL = 1$ signifie une prédiction parfaite (certitude totale) ; $PPL = |V|$ correspond au modèle aléatoire uniforme. L'objectif est de minimiser la PPL.

Perte cross-entropie	Perplexité	Interprétation qualitative
0.0	1.0	Prédiction parfaite (modèle idéal)
0.5	1.6	Modèle excellent
1.0	2.7	Très bon modèle

2.0	7.4	Modèle correct
3.5	33.1	Modèle faible
6.9	≈ 1 000	Modèle aléatoire uniforme ($ V = 1\,000$)

4.1.3 BPTT — Rétropropagation à travers le temps

La rétropropagation à travers le temps (BPTT) déroule le réseau récurrent dans le temps et applique la règle de la chaîne de manière récursive sur les T étapes temporelles. Le gradient de la perte totale par rapport aux paramètres W est :

$$\partial L / \partial W = \sum_{t=1}^T \partial L_t / \partial W$$

Chaque terme $\partial L_t / \partial W$ implique un produit de t matrices Jacobiennes. Si les valeurs propres de la Jacobienne $\partial h_t / \partial h_{t-1}$ sont inférieures à 1, le produit décroît exponentiellement (gradients évanescents — vanishing gradients), rendant impossible l'apprentissage de dépendances longue distance. Si elles sont supérieures à 1, le produit croît exponentiellement (gradients explosifs — exploding gradients), causant des instabilités numériques.

4.1.4 Gradient clipping

Le gradient clipping est la solution empirique au problème des gradients explosifs. Il borne la norme L2 du gradient global g à un seuil maximal τ , préservant la direction tout en bornant l'amplitude :

$$\text{si } \|g\|_2 > \tau \text{ alors } g \leftarrow g \cdot \tau / \|g\|_2$$

Cette opération n'affecte pas les mises à jour lorsque le gradient est en dessous du seuil. Elle évite les instabilités numériques (NaN dans les poids) sans dégrader la convergence normale. Une valeur de $\tau = 1.0$ est utilisée dans ce projet.

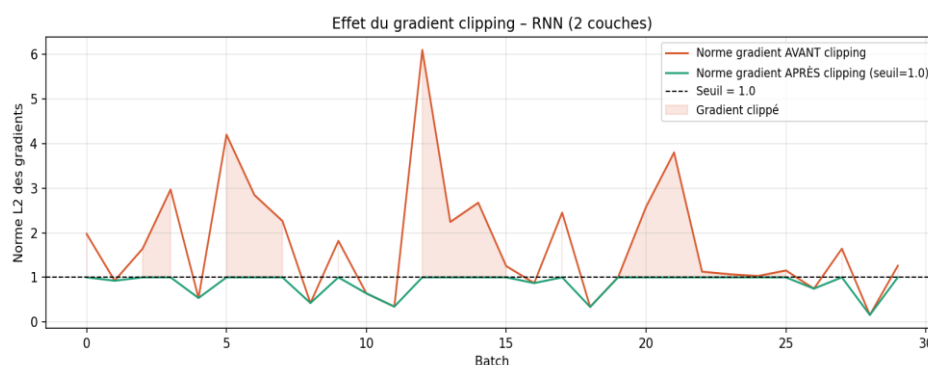


Figure 5 — Normes des gradients avant et après gradient clipping — RNN 2 couches (seuil = 1.0)

4.2 Architectures récurrentes

4.2.1 RNN simple

Le réseau récurrent simple maintient un état caché h_t résumant l'historique de la séquence :

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

Bien que conceptuellement élégant, le RNN simple souffre du problème des gradients évanescents pour les séquences longues. La multiplication répétée par la matrice W_h lors de la rétropropagation provoque une décroissance exponentielle du gradient, empêchant l'apprentissage de dépendances au-delà de 5 à 10 tokens de distance.

4.2.2 LSTM — Long Short-Term Memory

Le LSTM (Hochreiter & Schmidhuber, 1997) résout ce problème en introduisant une cellule mémoire c_t séparée de l'état caché h_t , contrôlée par trois portes différentiables :

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) && [\text{oubli}] \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) && [\text{entrée}] \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) && [\text{candidat}] \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && [\text{mémoire}] \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) && [\text{sortie}] \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

La clé du succès du LSTM est le chemin additif du gradient à travers c_t : lorsque $f_t \approx 1$ (aucun oubli), le gradient se propage sans atténuation multiplicative à travers les étapes temporelles, résolvant les gradients évanescents.

4.2.3 GRU — Gated Recurrent Unit

Le GRU (Cho et al., 2014) simplifie le LSTM en fusionnant les portes d'oubli et d'entrée en une porte de mise à jour z_t :

$$\begin{aligned} z_t &= \sigma(W_z[h_{t-1}, x_t]) && [\text{mise à jour}] \\ r_t &= \sigma(W_r[h_{t-1}, x_t]) && [\text{réinit.}] \\ \tilde{h}_t &= \tanh(W[h_{t-1}, x_t]) \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned}$$

Le GRU utilise environ 25% moins de paramètres que le LSTM pour des performances souvent comparables, le rendant plus efficace en termes de temps d'entraînement et de mémoire.

4.2.4 Tableau comparatif

Critère	RNN simple	LSTM	GRU
Mémoire	Court terme	Long + court terme	Long + court terme

Portes différentiables	Aucune	3 (oubli, entrée, sortie)	2 (update, reset)
Cellule mémoire séparée	Non	Oui (ct)	Non (fusionnée)
Paramètres (relatif)	1×	≈ 4×	≈ 3×
Gradients évanescents	Problème majeur	Résolu via ct	Partiellement résolu
Vitesse d'entraînement	Rapide	Plus lente	Intermédiaire
Usage recommandé	Séquences courtes	Longues dépendances	Compromis perf./coût

4.3 Résultats — Comparaison RNN / LSTM / GRU

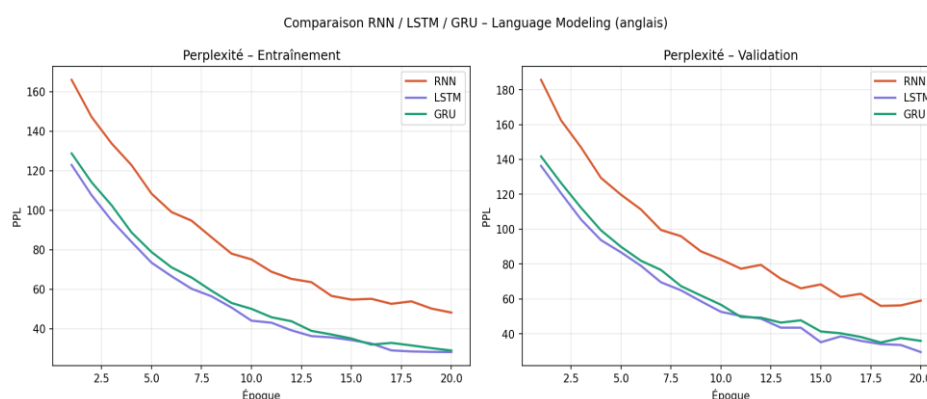


Figure 6 — Perplexité validation — comparaison RNN vs LSTM vs GRU (20 époques, corpus anglais)

Modèle	Paramètres	PPL val finale	Réduction vs RNN	Convergence
RNN simple	1 192 388	59.0	—	Instable, oscille
GRU	1 652 164	35.9	−39%	Stable
LSTM ✓	1 882 052	29.5	−50%	Stable, plus lente

Le RNN simple stagne avec une PPL élevée (59.0) et présente des oscillations importantes sur la courbe de validation — signe d'instabilité liée aux gradients explosifs partiellement maîtrisés par le clipping. Le LSTM atteint la meilleure perplexité (29.5, −50% vs RNN), démontrant l'efficacité de sa cellule mémoire pour les dépendances longue distance en anglais. Le GRU offre un excellent compromis (35.9) avec 12% moins de paramètres que le LSTM.

4.4 Système Seq2Seq

4.4.1 Motivation et architecture

Le RNN simple ne peut traiter que des paires séquence-séquence de même longueur. Pour la traduction automatique, les phrases source et cible ont des longueurs différentes et une structure non-isomorphe. Le schéma encodeur-décodeur (Cho et al., 2014 ; Sutskever et al., 2014) découple les deux phases :

- ▶ Encodeur GRU : lit la phrase source token par token et produit un vecteur contexte c défini comme l'état caché final h_T . Ce vecteur encapsule la signification sémantique de la phrase source entière en un vecteur de dimension fixe.
- ▶ Décodeur GRU : génère la traduction token par token en mode autorégressif. À chaque étape t , il reçoit le token précédent (ou le vrai token en mode teacher forcing) et l'état caché courant, et prédit la distribution de probabilité sur le vocabulaire cible.

4.4.2 Teacher Forcing et exposure bias

Le teacher forcing fournit le vrai token précédent y_{t-1} au décodeur pendant l'entraînement, accélérant la convergence mais créant un décalage d'exposition (exposure bias) : à l'inférence, le décodeur reçoit ses propres prédictions potentiellement erronées. Un ratio de teacher forcing dégressif ($0.8 \rightarrow 0.3$) atténue ce problème.

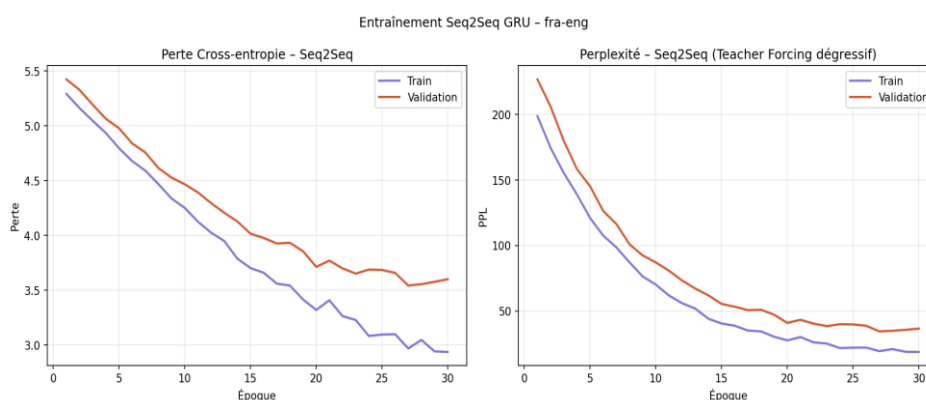


Figure 7 — Courbes d'entraînement Seq2Seq GRU — perte et perplexité (30 époques, fra-eng)

4.4.3 Stratégies de décodage

Décodage glouton (Greedy Search) : à chaque étape, le token de probabilité maximale est sélectionné : $\hat{y}_t = \arg\max P(v|\text{contexte})$. Simple et rapide ($O(T)$), mais myope — un mauvais choix précoce est irréversible.

Beam Search : maintient $k=3$ hypothèses en parallèle avec une length penalty ($\alpha=0.7$) pour éviter de favoriser les séquences courtes. Complexité $O(k \times |V| \times T)$, mais qualité significativement supérieure.

4.4.4 Résultats BLEU

Métrique	Greedy Search	Beam Search (k=3)	Gain absolu	Gain relatif
BLEU-1	74.80 %	84.74 %	+9.94 pts	↑ +13.3%

BLEU-2	43.99 %	64.82 %	+20.83 pts	↑ +47.4%
PPL val finale	36.6	36.6	—	—

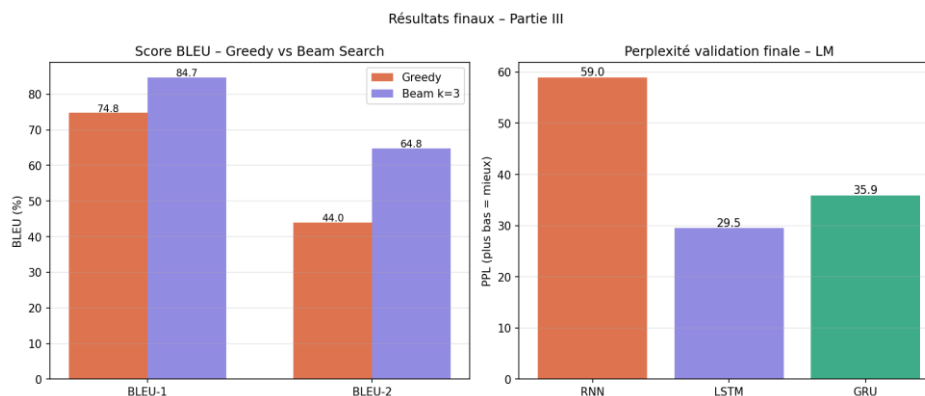


Figure 8 — Scores BLEU — Greedy Search vs Beam Search ($k=3$) et PPL finale par modèle

4.5 Synthèse — Partie III

Cette troisième partie démontre la progression naturelle des architectures pour la modélisation de séquences. Le RNN simple est limité par les gradients évanescents ($PPL = 59.0$). Le LSTM résout ce problème via la cellule mémoire et ses trois portes différentiables ($PPL = 29.5$, -50%). Le GRU offre un bon compromis ($PPL = 35.9$, -12% paramètres vs LSTM). Le Seq2Seq permet des tâches séquence-à-séquence de longueurs différentes, avec le Beam Search améliorant significativement la qualité des traductions ($BLEU-1 = 84.7\%$, $+9.9$ pts vs Greedy).

La limite principale du Seq2Seq sans attention est le goulot d'étranglement du vecteur contexte de taille fixe : toute l'information de la phrase source est compressée dans un vecteur de dimension 256, quelle que soit la longueur de la phrase. Cette observation est la motivation fondamentale du mécanisme d'attention (Bahdanau et al., 2015), qui permet au décodeur de consulter dynamiquement l'ensemble des états cachés de l'encodeur, et constitue la fondation des architectures Transformer (Vaswani et al., 2017).

Chapitre 5 — Discussion transversale

Question transversale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

5.1 Le paradigme commun

Les trois familles — MLP, CNN et RNN/Seq2Seq — partagent le même paradigme d'apprentissage supervisé : passe avant (calcul de $\hat{y}$), calcul de la perte, passe arrière (calcul des gradients par rétropropagation), et mise à jour des paramètres par l'optimiseur. Ce cadre unique est la force du deep learning : un seul ensemble d'outils théoriques et computationnels s'adapte à des modalités radicalement différentes. Ce qui diffère entre les architectures, ce sont les prior inductifs encodés dans leur structure — les hypothèses sur la géométrie des données qui contraignent l'espace des fonctions apprises.

5.2 Prior inductifs et géométrie des données

5.2.1 Données tabulaires → connexion totale (MLP)

Les données tabulaires sont des vecteurs hétérogènes sans structure spatiale ni temporelle. Chaque feature peut potentiellement interagir avec toutes les autres de manière non-linéaire. Le MLP encode l'hypothèse de connexion totale, maximale en l'absence d'information structurelle a priori. Cette généralité a un coût : en ne contraignant pas l'espace de recherche, le MLP nécessite plus de données pour converger vers des solutions généralisables.

5.2.2 Images → structure spatiale locale (CNN)

Les images ont une structure spatiale bien définie : les pixels voisins sont fortement corrélés, les features discriminantes sont locales (bords, textures, formes), et les objets peuvent apparaître à différentes positions. Le CNN encode ces trois propriétés via la localité du champ récepteur, le partage des poids (invariance à la translation), et la hiérarchie des couches. Cette adéquation permet +4.6 pts d'accuracy avec 5× moins de paramètres que le MLP sur Fashion-MNIST.

5.2.3 Séquences → dépendances temporelles (LSTM/Seq2Seq)

Les séquences textuelles ont une structure temporelle : l'ordre des tokens est crucial, et des dépendances peuvent exister entre tokens distants. Le LSTM encode l'hypothèse de dépendance temporelle via l'état caché, avec une mémoire sélective contrôlée par des portes différentiables. Le Seq2Seq étend cette architecture aux tâches à longueurs variables. Cette adéquation permet une PPL de 29.5 (vs 59.0 pour le RNN sans mémoire sélective).

5.3 Récapitulatif comparatif global

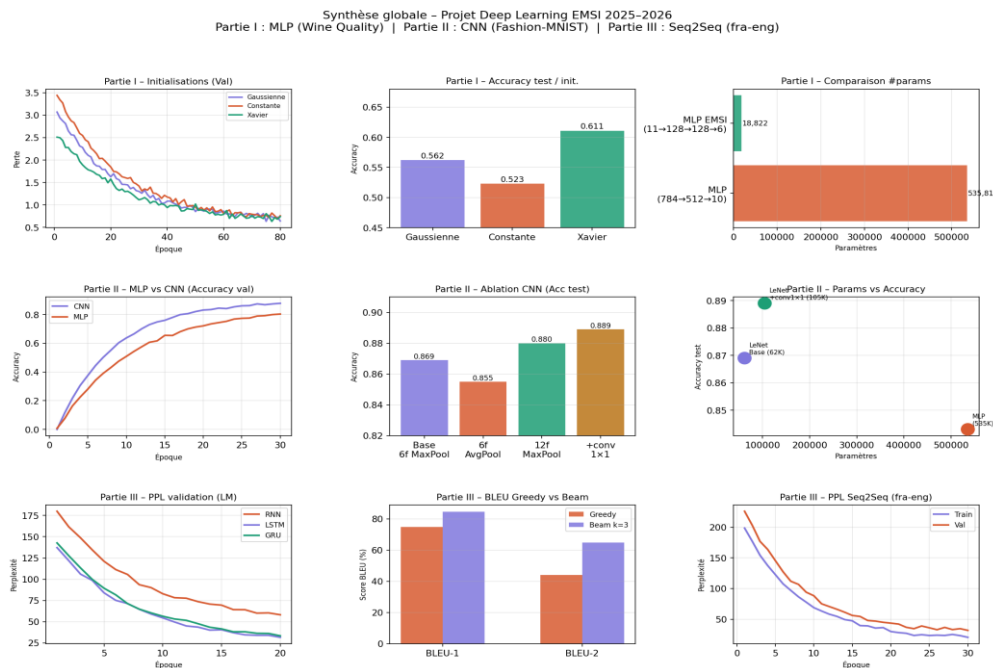


Figure 9 — Synthèse globale — résultats comparatifs des trois parties du projet

Critère	Partie I — MLP	Partie II — CNN	Partie III — LSTM/Seq2Seq
Type de données	Tabulaire (hétérogène)	Images 28×28	Séquences textuelles
Prior inductif	Connexion totale	Localité + partage poids	Dépendances temporelles
Architecture	11→128→128→6	LeNet + conv 1×1 + BN	GRU/LSTM 2 couches
Paramètres	18 822	104 526	1 882 052
Résultat principal	Acc = 0.611 F1 = 0.598	Acc = 0.889 (+4.6 pts/MLP)	PPL = 29.5 BLEU-1 = 84.7%
Initialisation	Xavier uniforme	Xavier + BN	Kaiming (PyTorch défaut)
Régularisation	Dropout + early stopping	BN + Dropout	Clipping + TF dégressif

5.4 Limites et perspectives

5.4.1 Limites identifiées

Partie I : déséquilibre des classes (42.6% classe 5) dégradant le rappel sur les qualités 3 et 8 ; opacité du modèle sans importance de features native ; performances inférieures aux méthodes ensemblistes (XGBoost) sur ce type de données.

Partie II : architecture relativement simple par rapport à l'état de l'art (ResNet, EfficientNet) ; visualisation limitée aux premières couches de features.

Partie III : goulot d'étranglement du vecteur contexte de taille fixe pour les phrases longues ; taille de corpus limitée (12k paires) insuffisante pour un système de traduction industriel.

5.4.2 Perspectives

- ▶ Mécanisme d'attention (Bahdanau, 2015) : consulter tous les états encodeurs à chaque étape de décodage, éliminant le goulot d'étranglement
- ▶ Architecture Transformer (Vaswani, 2017) : basée entièrement sur l'attention multi-têtes, état de l'art en NLP depuis 2018
- ▶ Transfer Learning : BERT/GPT pour le texte, ResNet/ViT pour les images — réduisant les données nécessaires
- ▶ Interprétabilité : SHAP (MLP), Grad-CAM (CNN), analyse des poids d'attention (Seq2Seq)
- ▶ Hyperparameter tuning automatique : Optuna, Ray Tune

Chapitre 6 — Conclusion générale

6.1 Bilan des contributions

Ce projet a mis en pratique l'ensemble des compétences fondamentales du deep learning à travers trois architectures complémentaires, démontrant systématiquement l'importance d'adapter le modèle à la structure des données.

Partie	Architecture	Dataset	Résultats principaux
I — MLP	11→128→128→6 (18 822 params)	Wine Quality Red	Acc=0.611 F1=0.598 Xavier optimal
II — CNN	LeNet+BN+conv1×1 (104 526 params)	Fashion-MNIST	Acc=0.889 (+4.6 pts/MLP, ×5 moins params)
III — LSTM	2 couches (1.88M params)	Corpus anglais	PPL=29.5 (−50% vs RNN simple)
III — Seq2Seq	GRU enc-dec + Beam Search	fra-eng	BLEU-1=84.7% (+9.9 pts vs Greedy)

6.2 Apports pédagogiques

Sur le plan de l'ingénierie PyTorch, ce projet a consolidé la maîtrise des briques fondamentales : structuration modulaire via `nn.Module`, inspection des paramètres via `named_parameters()` et `state_dict()`, sauvegarde disciplinée des modèles, gestion explicite du device, et pipelines de données avec `DataLoader`. Sur le plan théorique, les expériences ont permis de vérifier empiriquement plusieurs résultats importants : la supériorité de Xavier sur les initialisations naïves, l'effet stabilisant du gradient clipping sur l'entraînement du RNN, et le gain significatif du Beam Search sur le décodage glouton.

6.3 Réponse à la question directrice

Réponse synthétique

Le deep learning adapte ses architectures en encodant des prior inductifs différents selon la géométrie des données. Le MLP (connexion totale) convient aux données tabulaires sans structure. Le CNN (localité + partage de poids + hiérarchie) convient aux données à structure spatiale locale. Le LSTM (dépendances temporelles via portes différentiables) convient aux séquences à longues dépendances.

Ces prior réduisent l'espace des fonctions à apprendre, améliorent la généralisation et réduisent les besoins en données. Ne pas adapter l'architecture coûte cher : MLP sur images → 0.843 avec 535k params ; CNN adapté → 0.889 avec 104k params (+4.6 pts, ×5 moins de paramètres).

6.4 Conclusion

Ce projet illustre l'un des principes fondamentaux du machine learning moderne : la connaissance de la structure des données est un levier de performance aussi important que la puissance computationnelle. Les architectures du deep learning sont des formalisations mathématiques d'hypothèses sur le monde, construites et validées par des décennies de recherche. Comprendre ces hypothèses, savoir les choisir et les combiner, est au cœur du métier d'ingénieur en intelligence artificielle.

Bibliographie

Articles scientifiques fondateurs

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder. *EMNLP 2014*.
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- [5] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS 2010*, 249–256.
- [6] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS 2014*.
- [7] Vaswani, A. et al. (2017). Attention is All You Need. *NeurIPS 2017*.
- [8] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15(1), 1929–1958.
- [9] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training. *ICML 2015*.
- [10] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- [11] Papineni, K. et al. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. *ACL 2002*.

Datasets

- [12] Cortez, P. et al. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), 547–553. *UCI ML Repository*.
- [13] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*.
- [14] Tatoeba Project (2024). Open multilingual sentence database. <https://tatoeba.org> — Dataset fra-eng via www.manythings.org/anki/

Ouvrages et documentation

- [15] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
<https://www.deeplearningbook.org/>
- [16] Zhang, A. et al. (2023). Dive into Deep Learning. Cambridge University Press. <https://d2l.ai/>
- [17] PyTorch Documentation (2024). <https://pytorch.org/docs/stable/>

Annexes

Annexe A — Récapitulatif expérimental

3 Parties du projet	3 Datasets utilisés	9 Figures générées	19 Références biblio.
-------------------------------	-------------------------------	------------------------------	---------------------------------

A.1 Hyperparamètres Partie I (MLP)

Hyperparamètre	Valeur	Justification
Architecture	11→128→128→6	Compromis capacité/surapprentissage
Activation	ReLU	Évite gradient évanescent
Dropout	0.3	Régularisation efficace
Optimiseur	Adam (lr=1e-3)	Convergence rapide
Weight decay	1e-4	Régularisation L2 légère
Early stopping	Patience=20	Évite surapprentissage
Initialisation	Xavier uniforme	Meilleure convergence

A.2 Hyperparamètres Partie II (CNN)

Hyperparamètre	Valeur	Justification
Filtres Conv1/Conv2	12 / 16	Meilleur compromis
Padding	2 (same)	Préserve résolution 28×28
Pooling	MaxPool 2×2	+1.4 pts vs AvgPool
Conv 1×1	Oui	Meilleur ratio gain/coût
BatchNorm	Oui	Accélère convergence
Dropout FC	0.3	Régularisation
Augmentation	HFlip + RandomCrop	Généralisation

A.3 Hyperparamètres Partie III (RNN/Seq2Seq)

Hyperparamètre	Valeur	Justification
Dim. embedding	128	Standard pour vocab ~2500
Dim. état caché	256	Compromis capacité/coût
Couches récurrentes	2	Meilleure généralisation
Type cellule	GRU/LSTM	Bon compromis
TF init/final	0.8→0.3	Réduit exposure bias
Beam width	3	Compromis qualité/vitesse
Gradient clipping	1.0	Stabilise entraînement

Annexe B — Glossaire

Accuracy	Proportion de prédictions correctes sur le jeu de test
Attention	Mécanisme de pondération dynamique des états encodeurs lors du décodage
Backpropagation	Algorithme de calcul des gradients par règle de la chaîne récursive
Batch Normalization	Normalisation des activations dans un mini-batch pour accélérer la convergence
Beam Search	Décodage maintenant k hypothèses en parallèle pour maximiser la qualité globale
BLEU	Bilingual Evaluation Understudy — métriques de traduction basées sur les n-grammes
Dropout	Désactivation aléatoire de neurones pendant l'entraînement pour la régularisation
Gradient clipping	Bornage de la norme des gradients pour éviter les explosions
Gradient évanescent	Décroissance exponentielle des gradients dans les réseaux profonds/récurrents
LSTM	Long Short-Term Memory — cellule récurrente avec 3 portes différentiables
MLP	Multi-Layer Perceptron — réseau entièrement connecté
nn.Module	Classe de base PyTorch pour tous les réseaux de neurones
Perplexité	$\exp(\text{perte CE})$ — métrique d'évaluation des modèles de langage
Prior inductif	Hypothèse sur la structure des données encodée dans l'architecture
ReLU	$\max(0, x)$ — fonction d'activation non-linéaire standard
RNN	Recurrent Neural Network — réseau traitant des séquences via un état caché
Seq2Seq	Sequence-to-Sequence — architecture encodeur-décodeur pour la traduction
State dict	Dictionnaire des valeurs des paramètres d'un modèle PyTorch

Teacher Forcing	Fourniture du vrai token précédent au décodeur pendant l'entraînement
Xavier	Initialisation maintenant la variance constante à travers les couches (Glorot 2010)

Annexe C — Repository GitHub

Repository public du projet

URL : <https://github.com/salaheddine-elberrani/Deep-Learning-EMSI-2025>

Structure des dossiers :

Deep-Learning-EMSI-2025/

- └─ README.md ← Documentation complète du projet
- └─ notebooks/
 - | └─ Partie_I_MLP_PyTorch.ipynb ← MLP sur Wine Quality Red
 - | └─ Partie_II_CNN_Vision.ipynb ← CNN sur Fashion-MNIST
 - | └─ Partie_III_RNN_Seq2Seq.ipynb ← RNN/LSTM/GRU/Seq2Seq
- └─ figures/ ← 9 figures PNG haute résolution
- └─ docs/ ← Rapport DOCX + Présentation PPTX